

Model Behavior and Make It Testable

User Story • Use Case Lite • Acceptance Criteria

ทฤษฎี 2 ชั่วโมง + Lab/Workshop 3 ชั่วโมง
สำหรับนักศึกษาวิศวกรรมซอฟต์แวร์ชั้นปีที่ 2

เป้าหมายวันนี้
อธิบายพฤติกรรมที่ **trace** กลับไปยังหลักฐาน และตรวจได้



คำถามนำ: ใครทำอะไร ระบบตอบอย่างไร และผ่านอย่างไร?

เริ่มจาก **requirement** ที่พร้อม แล้วทำให้ความหมายต่อเนื่องไปถึงสิ่งที่ **tester** สังเกตได้

วันนี้ไม่ใช่

- full SRS
- screen design หรือ click path
- database / architecture

วันนี้คือ

- User Story = คุณค่าของ role
- Use Case Lite = พฤติกรรม actor/system
- Acceptance Criteria = ผลที่ตรวจได้

คำตอบต้องไม่เพิ่ม **fact, policy** หรือ **stakeholder** ใหม่

Week 05 ส่ง backlog ที่พร้อมให้ Week 06 ทำเป็น model

One meaning-preserving chain



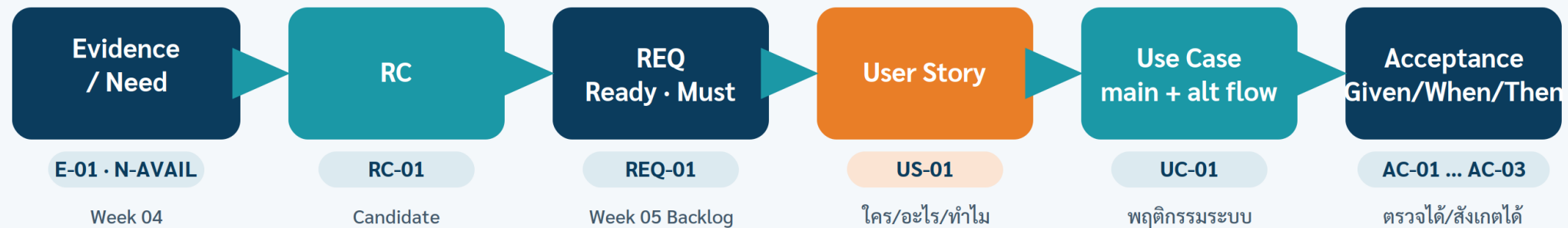
ทุกลูกศรต้องรักษาความหมายเดิม; ถ้าต้องเพิ่ม fact/rule ให้ย้อนกลับไป validate

Week 05: ID + source + priority + status → Week 06: story + behavior + pass/fail → Week 07: SRS Draft v1

Week 05 ส่ง backlog ที่พร้อมให้ Week 06 ทำเป็น model

สายการสืบย้อน (Trace Chain) — อย่างน้อย 1 เส้นต่อทีม

ทุกสิ่งก็ต้องรักษาความหมาย · ห้ามเพิ่ม actor / business rule ที่ไม่มีในต้นทาง



กติกาการ trace

- 1) ทุก US / UC / AC ต้องชี้กลับ REQ ได้
 - 2) REQ ต้องชี้กลับ Evidence/Need ได้
 - 3) หากพบข้อเท็จจริงใหม่ระหว่างทำ model ให้บันทึกเป็น Open Question
- ไม่ใช้การ “สร้าง” ข้อกำหนดใหม่ · Priority สูงไม่ได้แปลว่าพร้อม หาก Status ยังเป็น Needs Follow-up / Hold

Week 06 = แปลง Ready requirement ชุดเล็กเป็น behavior model ที่ trace ได้และตรวจได้ (ยังไม่ใช่ SRS ทั้งฉบับ)

Week 05: ID + source + priority + status → Week 06: story + behavior + pass/fail → Week 07: SRS Draft v1

Core คือ minimum ที่ทุกทีมต้องทำให้ผ่าน

CORE • 10 คะแนน

- User Stories 2 เรื่อง
- Use Case Lite 1 เรื่อง
- Main Flow 5–7 steps
- Alternate Flow อย่างน้อย 1
- Acceptance Criteria 3 ข้อ
- trace อย่างน้อย 1 chain

EXTENSION • หลัง Core ผ่าน

- story splitting
- fully dressed use case
- include / extend
- multiple models
- BDD automation

Extension เพิ่มได้ แต่ห้ามแทน **Core** ที่ยังไม่ครบ

Admission gate: เลือก requirement ที่พร้อม model

ไม่ใช่เลือกเพราะเป็น **Must** อย่างเดียว

1

มี **REQ-ID** และ **source trace**

stable identity

2

actor/beneficiary และ **goal** ชัด

ไม่ต้องเดาบทบาทใหม่

3

status Ready หรือมี **uncertainty** ที่ระบุชัด

ไม่เอา unknown มาแต่งเป็น rule

หยุดและถาม
หากไม่มี **Ready REQ**
หรือ **evidence** ไม่รองรับ

01

User Story

บอกว่า ใครต้องการอะไร เพื่ออะไร



องค์ประกอบ 3 ส่วน

ในฐานะ [role] ฉันต้องการ [goal] เพื่อ [value]

ส่วน	ตอบคำถาม	ตัวอย่าง (Campus Resource Booking)
role (ใคร)	ได้ประโยชน์ / เป็น actor จริง	Student Requester
goal (อะไร)	ต้องการทำอะไรเชิงคุณค่า	ค้นหาทรัพยากรที่ว่างตามวัน/เวลา
value (เพื่ออะไร)	ทำไมถึงมีคุณค่า	เลือกตัวเลือกก่อนส่งคำขอ

ระวัง: User Story ไม่ใช่ Use Case แบบย่อ — story ตอบ ใคร/อะไร/ทำไม, use case อธิบายปฏิสัมพันธ์เป็นขั้น

User Story กับ Use Case Lite — ใช้คู่กัน

User Story ≠ Use Case แบบย่อ

สองสิ่งนี้ตอบคนละคำถาม และใช้คู่กัน ไม่ใช่แทนกัน

User Story

“ใคร อยากรได้อะไร ทำไม”

ในฐานะ [role]
ฉันต้องการ [goal]
เพื่อ [value]

ให้อะไร

- มุมมองคุณค่าของผู้ใช้ · หน่วยเล็ก · จัดลำดับได้
- เป็นบทสนทนา ไม่ใช่ spec ของ flow

ตัวอย่าง

ในฐานะ Student Requester ฉันต้องการค้นหา
ทรัพยากรว่างตามวัน/เวลา เพื่อเลือกก่อนส่งคำขอ

+

Use Case (Lite)

“ปฏิสัมพันธ์เป็นขั้น”

Actor · Goal · Precondition · Trigger
Main Flow 5–7 ขั้น (actor / system)
Alternate Flow ≥ 1 (step + condition)
แสดง “ระบบตอบสนองอย่างไร” ที่ละขั้น

ให้อะไร

- ลำดับพฤติกรรมที่ตรวจได้ · เห็นทางปกติ+ข้อยกเว้น
- เป็นฐานให้ acceptance criteria

ตัวอย่าง

UC-01 “ค้นหาทรัพยากรว่าง” — ผู้ใช้ระบุเงื่อนไข ->
ระบบตรวจ/กรอง/แสดงผล -> ผู้ใช้เลือกหนึ่งรายการ

หนึ่ง story อาจถูกอธิบายด้วยหนึ่ง use case flow และตรวจด้วยหลาย acceptance criteria

User Story บอกคุณค่าของ role ไม่ใช่ลำดับขั้น

ในฐานะ [Role] ฉันต้องการ [Goal] เพื่อ [Value]

Weak

“ในฐานะผู้ใช้
ฉันกดปุ่มค้นหา”

Improved

“ในฐานะ Student Requester
ฉันต้องการค้นหาทรัพยากรว่าง
เพื่อเลือกตัวเลือกก่อนส่งคำขอ”

ถ้า **story** เริ่มเล่าหน้าจอหรือ **click** ให้ย้ายกลับไปหาผลลัพธ์ที่ **role** ต้องการ

Story ≠ Use Case

USER STORY

ใครต้องการผลลัพธ์อะไร
และผลลัพธ์นั้นมีคุณค่าอย่างไร

มุมมองคุณค่า

USE CASE LITE

actor และ system
โต้ตอบกันอย่างไรจนถึง goal
และเมื่อ condition เปลี่ยนจะเกิดอะไร

สัญญาพฤติกรรม

Story ไม่ใช่ Use Case ที่ย่อสั้นลง

02

Use Case Lite

อธิบายพฤติกรรมที่ละชั้น



หัวข้อของ Use Case (ตัวอย่าง UC-01)

ฟิลด์	ความหมาย	ตัวอย่าง
Use Case	ชื่อแบบ verb + object	ค้นหาทรัพยากรว่าง
Primary Actor	ผู้เริ่มและได้ประโยชน์	Student Requester
Goal	ผลลัพธ์ที่ต้องการ	เห็นรายการทรัพยากรที่ว่างตามเงื่อนไข
Preconditions (≤ 2)	เงื่อนไขก่อนเริ่ม	เข้าถึงระบบได้; availability ถูกโหลด
Trigger	เหตุการณ์เริ่มต้น	ต้องการหาทรัพยากรของวัน/ช่วงเวลาหนึ่ง

Main Flow ต้อง “ถูกระดับ” (5-7 ขั้น)

Main Flow ต้องอยู่ “ระดับที่ถูก”

สลับ Actor action / System response · 5-7 ขั้น · ไม่ใช่รายการปุ่ม/หน้าจอ

✓ ถูกระดับ — พฤติกรรมที่สังเกตได้

- 1 ผู้ใช้ระบุวัน เวลา และประเภททรัพยากรที่ต้องการ
- 2 ระบบตรวจว่าเงื่อนไขค้นหาครบและช่วงเวลาถูกต้อง
- 3 ระบบตรวจสอบสถานะว่างของทรัพยากรที่ตรงเงื่อนไข
- 4 ระบบตัดรายการที่ถูกยืนยันแล้วในช่วงเวลาซ้อนออก
- 5 ระบบแสดงรายการว่างพร้อมข้อมูลที่เป็นต่อการเลือก
- 6 ผู้ใช้เลือกทรัพยากรหนึ่งรายการเพื่อดำเนินการต่อ

Actor action

System response

✗ ผิดระดับ — เป็น UI steps

- ผู้ใช้คลิกปุ่ม “ค้นหา” สีฟ้า
- เปิด dropdown เลือกห้อง
- กรอกช่องวันที่ในฟอร์ม
- ระบบโหลด spinner แล้วดึง modal

เหตุผล: บอก “หน้าจอทำงานอย่างไร”
แทนที่จะบอก “ระบบทำอะไรให้ actor”
-> ผูกกับดีไซน์ ยังไม่ถึงเวลานี้

เคล็ดตรวจระดับ

ถ้าเปลี่ยนดีไซน์หน้าจอแล้วขั้นตอนยังจริงอยู่ = ถูกระดับ · ถ้าขั้นตอนพังเพราะเปลี่ยนปุ่ม/เมนู = เขียนต่ำเกินไป (UI step)

Alternate flow ต้องผูกกับ “step + condition” และบอก system response เสมอ (เช่น At step 5 · ไม่พบรายการว่าง)

Alternate / Exception Flow (อย่างน้อย 1)

ต้องผูกกับ step + condition ที่ชัด และบอก system response

At step: 5

Condition: ไม่พบทรัพยากรว่างตามวัน/เวลาที่ระบุ

System response: แจ้งว่าไม่พบรายการว่าง ให้ผู้ใช้แก้ไขเงื่อนไข โดยไม่แสดงทรัพยากรที่ถูกยืนยันแล้วว่า “ว่าง”

เริ่ม use case จาก actor, goal และ trigger

Use Case Lite 1 เรื่อง

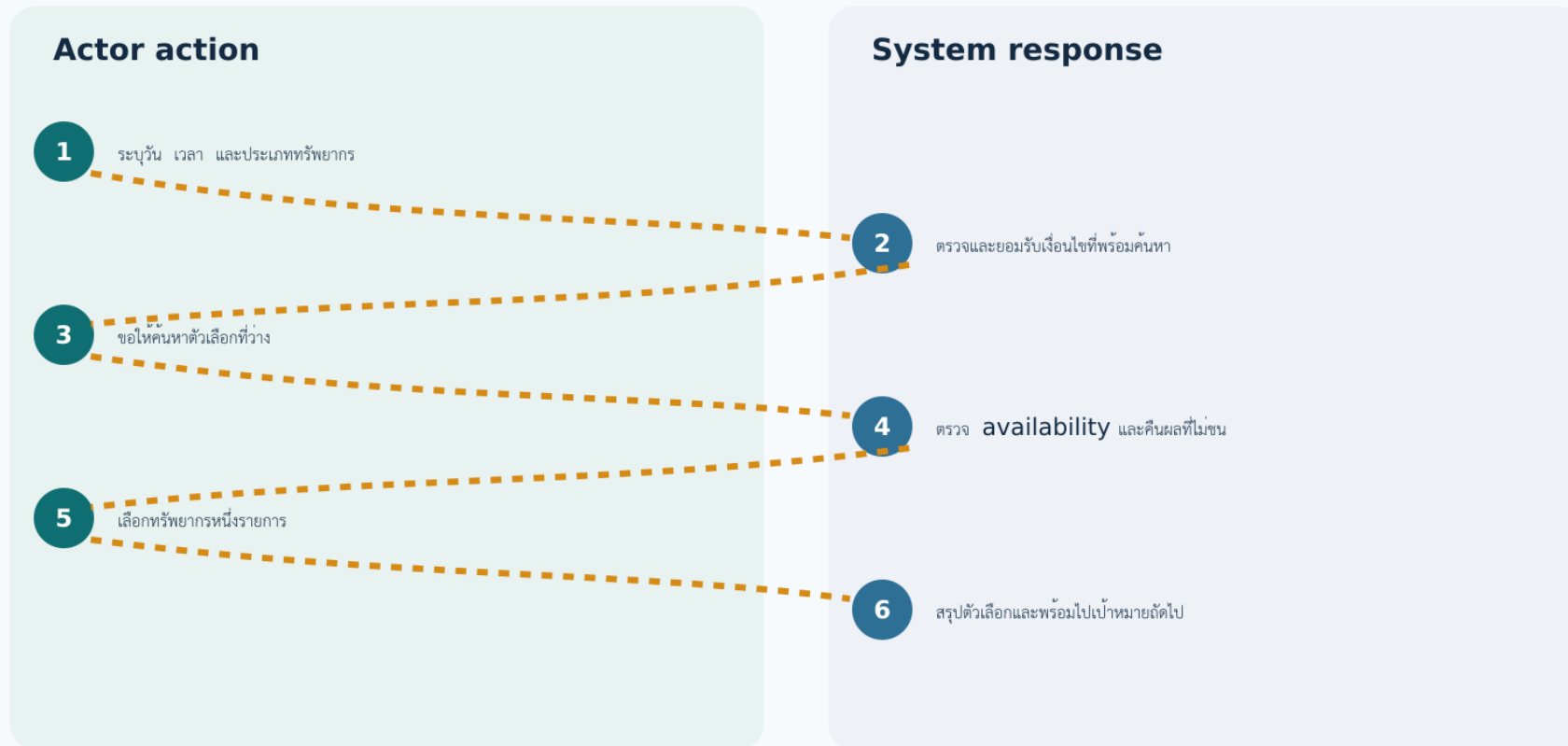
- ชื่อ: verb + object
- Primary actor: ผู้เริ่ม interaction
- Goal: ผลลัพธ์เมื่อจบ normal flow
- Preconditions: state ที่จำเป็น
- Trigger: เหตุที่เริ่ม use case

ถาม **stakeholder** เพื่อยืนยันพฤติกรรม
ไม่ใช่เพื่อสร้าง **policy** ใหม่



Main Flow ต้องเห็นการสลับ actor action / system response

Main Flow = behavior, not screen clicks



หลีกเลี่ยงคำว่า เปิดหน้า • คลิกเมนู • กดปุ่ม • query table

ตรวจให้ครบ

- 5–7 steps
- ทุก step มี lane
- ระบุ behavior
- ไม่ใช่ UI steps

พฤติกรรมไม่ใช่ **click path** หรือ **implementation**

เขียนได้

- actor ระบุเงื่อนไข
- system ตรวจสอบและคืนผล
- actor เลือกตัวเลือก

ยังไม่เขียนใน **Week 06**

- เปิดหน้าแรก / กดปุ่มสีเขียว
- query table booking
- เรียก microservice / ใช้ cache

อ่าน **flow** ออกเสียง: ถ้าไม่เกิด **interaction** ต่อกัน ให้กลับไปแก้

Alternate Flow ต้องผูกกับ **step** และ **condition** ที่ตรวจได้

At step 4	system ตรวจ availability
Condition	ไม่พบทรัพยากรที่ตรงวัน เวลา และประเภท
System response	แจ้งว่าไม่พบตัวเลือก และคืนเงื่อนไขให้ผู้ปรับ

จากนั้น **actor** ปรับเงื่อนไขและกลับ **Step 1** หรือจบ **flow**

Overview ≠ Written Flow

Use Case Overview Lite

Campus Resource Booking · actor + system boundary + 4 goals

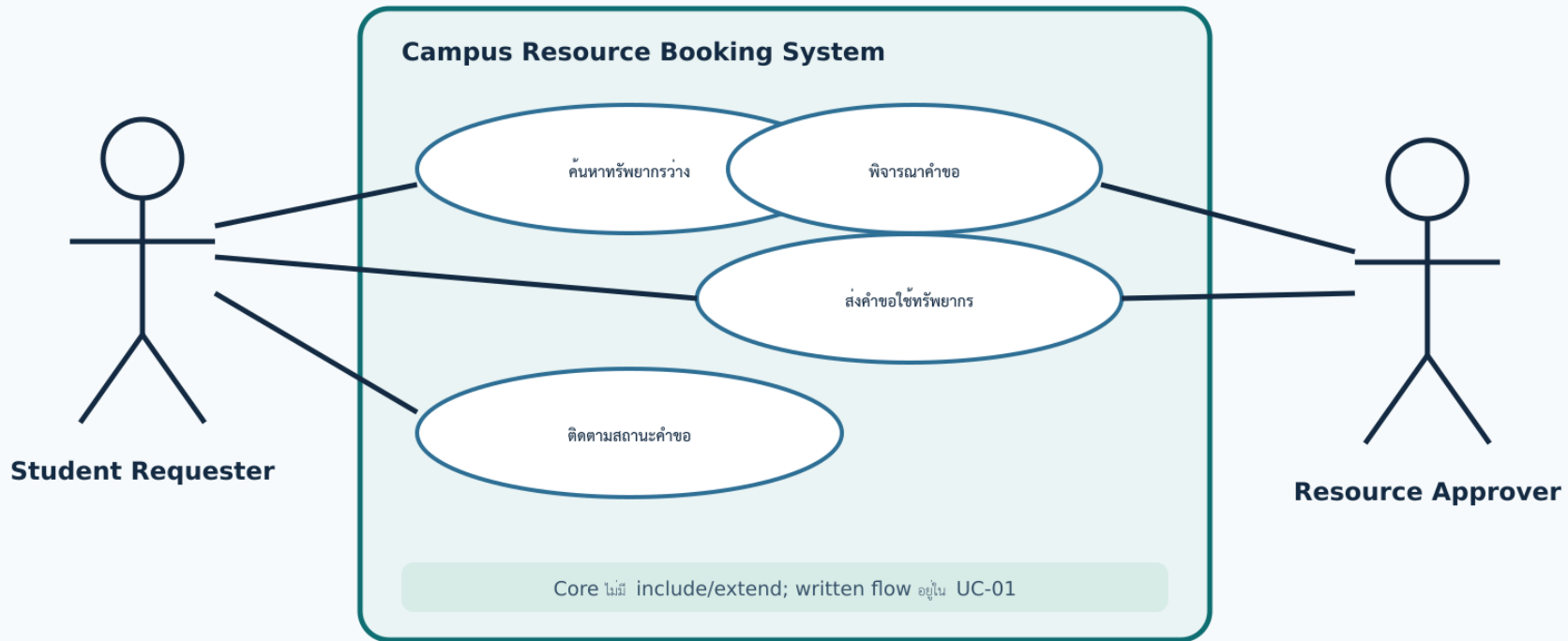


Diagram นี้ไม่แสดง UI, database หรือ architecture ภายในระบบ

Core diagram: system boundary + actors + 3–5 use cases ที่ source รองรับ

03

Acceptance Criteria

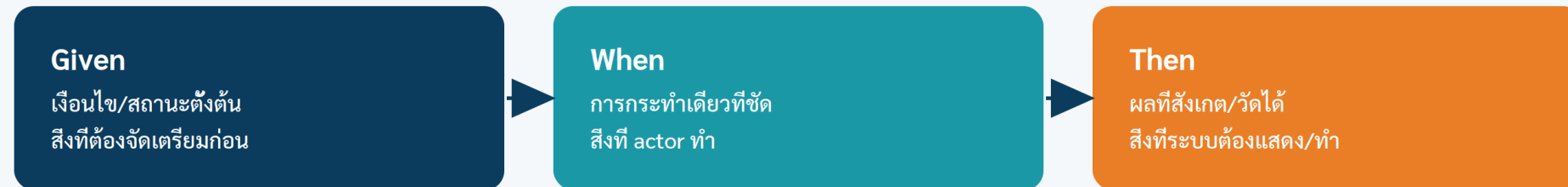
ทำให้ผลลัพธ์ตรวจได้



Given / When / Then

Acceptance Criteria = เงื่อนไข + การกระทำ + ผลที่สังเกตได้

รูปแบบ Given / When / Then ทำให้ผู้ทดสอบตัดสินใจ “ผ่าน/ไม่ผ่าน” ได้



ตัวอย่างที่ดี (AC-01 · trace REQ-01, REQ-04)

Given ห้อง A ถูกยืนยันแล้วเวลา 13:00–15:00

When ผู้ใช้ค้นหาช่วงเวลาเดียวกัน

Then ห้อง A ไม่ปรากฏในรายการ “ว่าง”

✗ กำรวม — วัดไม่ได้

“ระบบใช้งานง่าย”

“ค้นหาเร็ว” · “แสดงข้อมูลถูกต้อง”

ไม่มีเงื่อนไข ไม่มีผลที่สังเกตได้ -> เกือบไม่จบ

✓ สังเกตได้ — ตรวจได้

“ผลค้นหาแสดงเฉพาะรายการที่ยังไม่ถูกยืนยัน
ในช่วงเวลาที่เลือก”

มีเงื่อนไข + การกระทำ + ผลที่ผู้ทดสอบเห็นได้

ทุกข้อต้องมี เงื่อนไข (Given) + การกระทำ (When) + ผลที่สังเกตได้ (Then) ที่ตัดสิน “ผ่าน/ไม่ผ่าน” ได้

Acceptance Criteria = Pass / Fail

Given [condition/state] • When [action/event] • Then [observable result]

Vague

“ระบบแสดงข้อมูลถูกต้อง”

Observable

Given ห้อง A ถูกยืนยัน 13:00–15:00

When ค้นหาช่วงเดียวกัน

Then ห้อง A ไม่ปรากฏเป็นรายการว่าง

สังเกตได้ vs กำกวม

ดี — สังเกตได้ (AC-01)

Given ห้อง A ถูกยืนยันแล้ว 13:00–15:00

When ผู้ใช้ค้นหาช่วงเวลาเดียวกัน

Then ห้อง A ไม่ปรากฏในรายการ “ว่าง”

กำกวม — วัดไม่ได้

“ระบบใช้งานง่าย”

“ค้นหาเร็ว”

“แสดงข้อมูลถูกต้อง”

ไม่มีเงื่อนไข ไม่มีผลที่สังเกตได้ → เกียงกันไม่จบ ผู้ทดสอบตัดสินไม่ได้

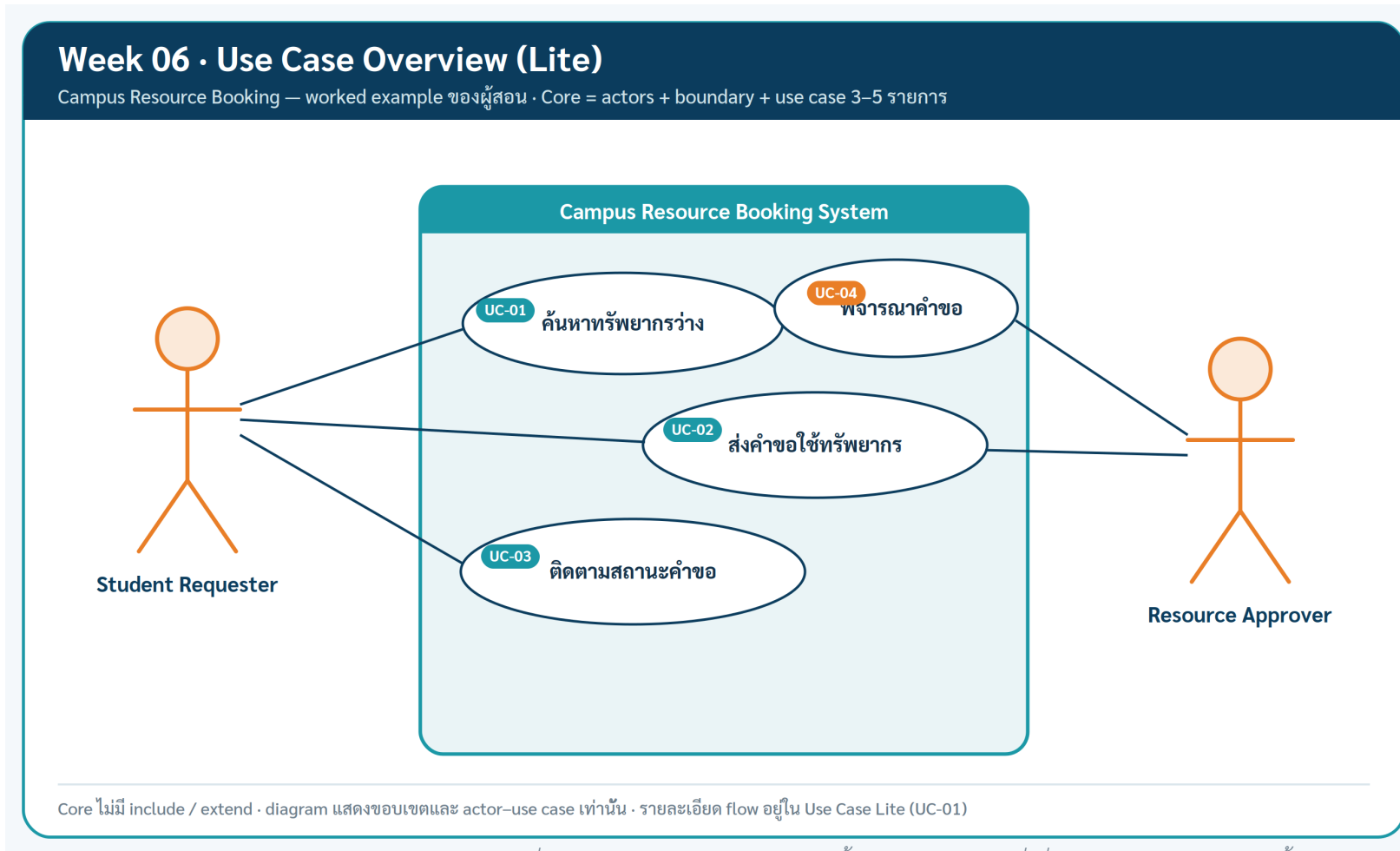
ทุก model ต้อง trace ได้อย่างน้อย 1 เส้น

Story / Use Case / AC	Requirement	Evidence
US-01 / ค้นหาทรัพยากรว่าง (UC-01) / AC-01-AC-03	REQ-01, REQ-04	E-01, E-03, F-03
US-02 / ส่งคำขอใช้ทรัพยากร	REQ-02	E-04

สายการสืบย้อน (ตัวอย่าง)

E-01 / N-AVAIL → RC-01 → REQ-01 [Ready, Must] → US-01
 → UC-01 (main 6 / alternate step 5) → AC-01...AC-03

Use Case Overview (Core) — Campus Resource Booking



actors + boundary + use case 3–5 · ไม่มี include/extend · diagram ต้องตรงกับ text · ไม่เพิ่ม actor/behavior นอกต้นทาง

เดินครบวงจร — Campus Resource Booking

Stories

US-01 ในฐานะ Student Requester ต้องการค้นหาทรัพยากรว่างตามวัน/เวลา เพื่อเลือกก่อนส่งคำขอ (REQ-01)

US-02 ในฐานะ Student Requester ต้องการส่งวัน/เวลา/ทรัพยากร/วัตถุประสงค์ เพื่อให้คำขอเข้าสู่การพิจารณา (REQ-02)

UC-01 “ค้นหาทรัพยากรว่าง” · Main Flow (6)

1 ระบุวัน/เวลา/ประเภท → 2 ระบบตรวจสอบครบ → 3 ตรวจสอบ availability → 4 ตัดที่ยืนยันซ้อน → 5 แสดงรายการว่าง →
6 ผู้ใช้เลือก · Alternate @5: ไม่พบว่าง

Acceptance Criteria

AC-01	Given ห้อง A ยืนยัน 13:00–15:00 · When ค้นหาช่วงเดียวกัน · Then ห้อง A ไม่อยู่ใน “ว่าง”	REQ-01,04
AC-02	Given ห้อง B ว่างตรงประเภท · When ค้นหา · Then ห้อง B ปรากฏพร้อมวัน/เวลา	REQ-01
AC-03	Given ยังไม่ระบุเวลา · When กดค้นหา · Then ไม่ค้นหาและชี้ช่องที่ต้องกรอก	REQ-01

ความเข้าใจผิดที่ต้องแก้

ความเข้าใจผิด	วิธีคิดที่ถูก
User Story = Use Case แบบย่อ	คนละคำถาม; story = ใคร/อะไร/ทำไม, UC = ปฏิสัมพันธ์เป็นขั้น
AC คือการทวน requirement	AC ต้องมี condition/action/observable result
“ง่าย/เร็ว/ถูกต้อง” เป็น AC ได้	ต้องเปลี่ยนเป็น scenario ที่สังเกตผลได้
Main flow = รายการปุ่ม/หน้าจอ	ต้องสลับ actor action/system response
Diagram แทน specification ได้	diagram เสริม text; ห้ามเพิ่ม actor/behavior นอกต้นทาง
Priority สูง = พร้อมใช้	ถ้า status Needs Follow-up/Hold ยังไม่พร้อมทำ model

Responsible AI + Two-Repository

READ จาก Course Repo -> WORK และ SUBMIT ใน Team Repo

ห้ามแก้ Course Repo · ห้ามใส่เฉลย/คะแนน/ไฟล์ผู้สอนลง Student Template

Course Repo — READ ONLY

- คำสั่งสัปดาห์ · rubric · assignment contract
 - template ของงาน
templates/06-requirement-models.md
 - completed example (Campus Resource Booking)
examples/campus-resource-booking/week-06/
 - Case Card 01–10 (ทีมใช้ Case ID ของตน)
 - prompt pack สำหรับ Responsible AI
- เจ้าของ/แก้ไข: ผู้สอนเท่านั้น

คัดลอก
ไม่แก้ต้นทาง

Team Repo — WORK & SUBMIT

- งานจริงของทีม (Case ID ของตนเอง)
docs/06-requirement-models.md
diagrams/use-case/w06-use-case-overview.svg
submissions/week-06-submission.md
 - evidence · worklog · decision log
 - AI use log + human verification
 - commit: **submit(w06): beginner requirement models**
- เจ้าของ/แก้ไข: ทีมนักศึกษา

AI = ผู้ช่วย ไม่ใช่แหล่งข้อเท็จจริง

- ใช้ช่วยตรวจ/ตั้งคำถาม/เสนอถ้อยคำได้
- ห้ามให้ AI แต่ง fact/actor/policy/rule ใหม่
- บันทึก AI use log: สั่งอะไร / ใช้-ปฏิเสธ / ตรวจกับ source ไດ / มนุษย์แก้อะไร
- ทุก output ต้องมนุษย์ยืนยันกับ evidence

READ จาก Course Repo → WORK/SUBMIT ใน Team Repo (ห้ามแก้ Course Repo / ห้ามใส่เฉลยลง Student Template)

Tester ต้องเตรียม **state, action** และสิ่งที่สังเกตได้



Prepare — Act — Observe

Prepare	state/data อะไรต้องมี
Act	action/event อะไรเกิดขึ้น
Observe	อะไรบอกว่า Pass หรือ Fail

หากยังไม่มี **evidence** สำหรับ **target** เช่น “เร็ว” ให้เก็บเป็น **open question**

หนึ่ง chain ที่ดี รักษาความหมายตั้งแต่หลักฐานถึง AC

One meaning-preserving chain



ทุกลูกศรต้องรักษาความหมายเดิม; ถ้าต้องเพิ่ม fact/rule ให้ย้อนกลับไป validate

E-01 / E-03 → REQ-01 → story: ค้นหาทรัพยากรว่าง → UC-01 → AC: ห้องที่ยืนยันแล้วไม่เป็นรายการว่าง

AI ช่วย simulate review ได้ แต่คนต้องยืนยันกับ source

AI may help

- ชี้ ambiguity หรือ trace gap
- เสนอคำถาม reviewer
- สรุปหลัง human verification

AI must not

- สร้าง case fact / policy ใหม่
- แทน evidence หรือ exit ticket
- กลบ uncertainty ให้ดูเหมือนยืนยันแล้ว

AI use log: purpose • prompt summary • output used/rejected • verification source • human change

Lab 3 ชั่วโมง: checkpoint ใน Team Repo

0–10

Preflight + Ready REQ

10–35

2 User Stories

35–80

Use Case Lite

80–115

Acceptance Criteria

115–150

Diagram + peer trace review

150–180

submission + Git + exit ticket

READ from Course Repo → WORK/SUBMIT in Team Repo

Peer review เดิน trace chain ไม่ใช่ตรวจคำสะกดอย่างเดียว

Reviewer ถาม 5 ข้อ

- role และ goal มาจาก source ไດ?
- step มี actor/system lane ครบไหม?
- alternate ผูกกับ condition และ step ไหม?
- Then สังเกตผล Pass/Fail ได้ไหม?
- diagram ตรงกับ written model ไหม?



บันทึก defect/confirmation และ before/after revision

Submission: path + staged diff

ส่งใน Team Repo

- docs/06-requirement-models.md
- diagrams/use-case/w06-use-case-overview.svg หรือ .png
- submissions/week-06-submission.md
- worklog + AI use log + exit ticket

`git diff --check → git add [exact paths] → commit → push`



แผน 5 ชั้น · 180 นาที

ชั้น	นาที	งานของทีม
Setup	10	READ course repo, copy template, สร้าง branch
1	35	เขียน 2 User Stories จาก top backlog
2	45	Use case main 5–7 + alternate ≥1
3	35	เขียน AC 3 แบบ Given/When/Then
4	35	Peer trace review + revise
5	30	Export diagram, submission, commit/push

ก่อนจบคาบ: เลือก AC 1 ข้อ ถามว่าผู้ทดสอบรู้ “ผ่าน/ไม่ผ่าน” อย่างไร แล้ว commit submit(w06): beginner requirement models

Definition of Done: ทำ Core ครบและบอกเหตุผลย้อนกลับได้

- 2 stories • 1 UC • main 5–7 • alternate ≥ 1 • AC 3
- trace อย่างน้อย 1 chain ถึง evidence
- diagram, relative links และ paths เปิดได้
- peer review / revision / AI disclosure / worklog / submission ครบ
- individual exit ticket: Prepare–Act–Observe–Trace

Final commit: submit(w06): beginner requirement models